



Lux Security Token Standard: ERC-1400 on Post-Quantum EVM with MPC Custody

Zach Kelling

Lux Industries

2025

Abstract

We present the Lux Security Token Standard, an implementation of ERC-1400 (Security Token Standard) on Lux EVM with post-quantum cryptographic precompiles and MPC threshold custody for controller operations. This standard evolves from Arca’s **st-contracts** (2018), which implemented ERC-1400/ERC-1404 security token protocols for ArCoin and the Arca Funds. The Lux implementation extends the original standard with: (i) transfer restriction enforcement via on-chain compliance oracles linked to I-Chain DIDs, (ii) post-quantum signature verification through ETHFALCON and Ringtail precompiles that protect long-lived securities tokens against quantum adversaries, (iii) MPC threshold signing for controller operations so that no single entity—not the issuer, not the platform, not any regulator—can unilaterally force-transfer securities, and (iv) partition management for complex capital structures including preferred/common partitions, lockup tranches, and vesting schedules. We provide formal specifications for all token operations, prove that the transfer restriction mechanism is sound and complete with respect to a given compliance policy, and compare the standard against ERC-3643 (T-REX), ERC-1404, and proprietary alternatives.

1 Introduction

Security tokens—blockchain-based representations of regulated securities—require capabilities far beyond those of standard fungible tokens. An ERC-20 token can be transferred freely between any two addresses. A security token must enforce transfer restrictions based on investor accreditation, jurisdictional rules, lockup periods, and regulatory holds. It must support forced transfers for court orders and regulatory actions. It must maintain auditable records of all ownership changes. It must manage complex capital structures through partitions.

1.1 Heritage: Arca **st-contracts** (2018)

The first production implementation of these requirements was Arca’s **st-contracts**, developed in 2018 for the ArCoin digital security. ArCoin was an interest-bearing token backed by US Treasury bills, requiring:

- **Transfer restrictions:** Only accredited investors in permitted jurisdictions could hold ArCoin.
- **Interest distribution:** Automated calculation and distribution of T-bill interest to token holders.
- **NAV tracking:** On-chain net asset value updates reflecting the underlying T-bill portfolio.
- **Regulatory compliance:** Compliance with SEC Regulation D, Rule 506(c) for accredited investor verification.
- **Forced transfers:** Ability to execute transfers mandated by court orders or regulatory actions.

The **st-contracts** implementation used ERC-1400 for the core security token interface and ERC-1404 for transfer restriction logic. This implementation was deployed on Ethereum mainnet and operated without incident for the lifetime of the ArCoin program.

1.2 Limitations of the Original Design

The Arca implementation, while functional, had several architectural limitations that motivated the Lux standard:

1. **Centralized controller:** A single Ethereum address controlled forced transfers and document management. Compromise of this key would grant full control over all securities.
2. **Static restrictions:** Transfer restriction rules were compiled into the contract and required redeployment to update.
3. **No quantum resistance:** ECDSA signatures protecting controller operations and investor wallets are vulnerable to Shor’s algorithm on future quantum computers.

4. **Limited partitioning:** The original ERC-1400 partition implementation did not support the complex capital structures required for private equity, real estate, and structured products.
5. **Gas costs:** On Ethereum mainnet, the gas costs for compliance-checked transfers made small-denomination trading economically impractical.

1.3 Contributions

This paper presents the Lux Security Token Standard, which addresses each limitation:

1. MPC threshold controller: Controller operations require t -of- n threshold signatures, eliminating single points of compromise [1, 2].
2. Dynamic compliance oracles: Transfer restrictions are evaluated by on-chain compliance oracles that reference I-Chain DID claims, enabling rule updates without contract redeployment.
3. Post-quantum precompiles: ETHFALCON and Ringtail signature verification is available as EVM precompiles, enabling quantum-resistant token operations [5].
4. Extended partitions: Support for hierarchical partitions with independent restriction rules, enabling complex capital structures.
5. Lux EVM execution: Low gas costs (sub-cent per transfer) make small-denomination trading viable.

2 Token Interface

2.1 Core ERC-1400 Interface

The Lux Security Token Standard implements the full ERC-1400 interface:

Listing 1: Core security token interface.

```

1 interface ISecurityToken is IERC20 {
2     // Document management
3     function getDocument(bytes32 name)
4         external view returns (string memory uri, bytes32 hash, uint256 timestamp);
5     function setDocument(bytes32 name, string calldata uri, bytes32 hash) external;
6     function getAllDocuments() external view returns (bytes32[] memory);
7
8     // Controller operations (MPC threshold required)
9     function isControllable() external view returns (bool);
10    function controllerTransfer(
11        address from, address to, uint256 value,
12        bytes calldata data, bytes calldata operatorData
13    ) external;
14    function controllerRedeem(
15        address holder, uint256 value,
16        bytes calldata data, bytes calldata operatorData
17    ) external;
18
19    // Partition management
20    function balanceOfByPartition(bytes32 partition, address holder)
21        external view returns (uint256);
22    function partitionsOf(address holder)
23        external view returns (bytes32[] memory);
24    function transferByPartition(
25        bytes32 partition, address from, address to,
26        uint256 value, bytes calldata data
27    ) external returns (bytes32);
28
29    // Transfer restriction
30    function canTransfer(address to, uint256 value, bytes calldata data)
31        external view returns (bytes1 statusCode, bytes32 appCode);
32    function canTransferByPartition(
33        bytes32 partition, address to, uint256 value, bytes calldata data
34    ) external view returns (bytes1 statusCode, bytes32 appCode, bytes32 partition);

```

```

35
36 // Issuance
37 function isIssuable() external view returns (bool);
38 function issue(address holder, uint256 value, bytes calldata data) external;
39 function issueByPartition(
40     bytes32 partition, address holder, uint256 value, bytes calldata data
41 ) external;
42
43 // Redemption
44 function redeem(uint256 value, bytes calldata data) external;
45 function redeemByPartition(
46     bytes32 partition, uint256 value, bytes calldata data
47 ) external;
48 }

```

2.2 ERC-1404 Transfer Restrictions

Transfer restriction checking extends ERC-1404 with a richer status code system:

Listing 2: Transfer restriction interface.

```

1 interface ITransferRestriction {
2     function detectTransferRestriction(
3         address from, address to, uint256 value
4     ) external view returns (uint8 restrictionCode);
5
6     function messageForTransferRestriction(uint8 restrictionCode)
7         external pure returns (string memory message);
8 }

```

Table 1: Transfer restriction codes.

Code	Name	Description
0x00	SUCCESS	Transfer permitted
0x01	SENDER_NOT_ELIGIBLE	Sender fails compliance check
0x02	RECEIVER_NOT_ELIGIBLE	Receiver fails compliance check
0x03	SENDER_LOCKUP	Sender’s tokens are in lockup period
0x04	RECEIVER_LIMIT	Receiver would exceed maximum holding
0x05	JURISDICTION_BLOCKED	Transfer blocked by jurisdictional rule
0x06	ACCREDITATION_EXPIRED	Receiver’s accreditation has expired
0x07	REGULATORY_HOLD	Tokens subject to regulatory hold/freeze
0x08	MAX_HOLDERS	Transfer would exceed maximum holder count
0x09	PARTITION_RESTRICTED	Partition-specific restriction applies
0x0A	KYC_INCOMPLETE	Receiver has not completed KYC
0x0B	AML_FLAG	AML screening flag on sender or receiver
0xFF	UNKNOWN	Unspecified restriction

3 Compliance Oracle

3.1 Architecture

Transfer restrictions are evaluated by on-chain compliance oracles rather than hardcoded contract logic. Each security token references a compliance oracle contract that implements the

restriction checking interface:

Listing 3: Compliance oracle.

```
1 contract ComplianceOracle is ITransferRestriction {
2     IIdentityRegistry public identityRegistry; // I-Chain DID registry
3     mapping(bytes32 => Rule[]) public partitionRules;
4     Rule[] public globalRules;
5
6     struct Rule {
7         RuleType ruleType;
8         bytes params;
9         bool active;
10    }
11
12    enum RuleType {
13        WHITELIST,           // only whitelisted addresses
14        ACCREDITATION,       // require valid accreditation credential
15        JURISDICTION,        // block/allow specific jurisdictions
16        LOCKUP,              // time-based transfer restriction
17        MAX_HOLDERS,         // cap on total holder count
18        MAX_BALANCE,         // cap on individual holding
19        MIN_TRANSFER,        // minimum transfer amount
20        VELOCITY,            // maximum transfer frequency
21    }
22
23    function detectTransferRestriction(
24        address from, address to, uint256 value
25    ) external view override returns (uint8) {
26        // Check global rules
27        for (uint i = 0; i < globalRules.length; i++) {
28            uint8 code = evaluateRule(globalRules[i], from, to, value);
29            if (code != 0x00) return code;
30        }
31        return 0x00; // SUCCESS
32    }
33
34    function evaluateRule(
35        Rule memory rule, address from, address to, uint256 value
36    ) internal view returns (uint8) {
37        if (!rule.active) return 0x00;
38
39        if (rule.ruleType == RuleType.ACCREDITATION) {
40            bytes32 did = identityRegistry.didOf(to);
41            if (!identityRegistry.isValidCredential(did, "accreditation")) {
42                return 0x06; // ACCREDITATION_EXPIRED
43            }
44        }
45        // ... additional rule evaluation
46        return 0x00;
47    }
48 }
```

3.2 I-Chain DID Integration

The compliance oracle queries the I-Chain identity registry to verify investor credentials. Each investor's DID contains verifiable credentials issued by KYC/AML providers (KYC providers integrated via I-Chain):

- **KYC credential:** Issued upon successful identity verification, contains verification level and expiry.
- **Accreditation credential:** Issued upon accredited investor verification per SEC Rule 501(a), contains basis (income, net worth, professional) and expiry date.
- **Jurisdiction credential:** Contains the investor's country and state/province of residence for jurisdictional rule evaluation.
- **AML credential:** Contains ongoing AML monitoring status, updated by continuous screening.

The oracle verifies credentials without accessing the underlying personal data, preserving investor privacy while enforcing compliance requirements.

3.3 Formal Properties

Definition 1 (Compliance Policy). *A compliance policy Π is a finite set of rules $\{r_1, r_2, \dots, r_k\}$ where each rule $r_i : (\text{Address} \times \text{Address} \times \mathbb{N}) \rightarrow \{0, 1\}$ maps a (sender, receiver, amount) triple to a boolean decision.*

Definition 2 (Sound Restriction). *A transfer restriction mechanism is sound with respect to policy Π if every transfer that the mechanism permits also satisfies Π : $\forall (s, r, v)$, if $\text{detectRestriction}(s, r, v) = 0$ then $\forall r_i \in \Pi, r_i(s, r, v) = 1$.*

Definition 3 (Complete Restriction). *A transfer restriction mechanism is complete with respect to policy Π if every transfer that satisfies Π is also permitted by the mechanism: $\forall (s, r, v)$, if $\forall r_i \in \Pi, r_i(s, r, v) = 1$ then $\text{detectRestriction}(s, r, v) = 0$.*

Theorem 1 (Soundness and Completeness). *The compliance oracle transfer restriction mechanism is sound and complete with respect to the configured compliance policy Π , provided that the I-Chain identity registry returns correct credential status.*

Proof. Soundness: `detectTransferRestriction` iterates over all rules in Π (both global and partition-specific). If any rule r_i evaluates to a non-zero restriction code, the function returns that code and the transfer is blocked. A transfer is permitted (code 0x00) only if every rule returns 0x00. By construction, code 0x00 is returned by `evaluateRule` only when the rule's condition is satisfied. Therefore, a permitted transfer satisfies all rules in Π .

Completeness: If all rules in Π are satisfied for a given transfer, then each call to `evaluateRule` returns 0x00. The loop produces no non-zero code. The function returns 0x00, permitting the transfer.

The correctness assumption on the I-Chain registry is necessary because the oracle delegates credential verification to the registry. If the registry returns incorrect status (e.g., reports an expired accreditation as valid), soundness may be violated. The I-Chain registry's correctness is maintained by the DID credential verification protocol. $\square$

4 Partition Management

4.1 Partition Types

The Lux standard extends ERC-1400 partitions to support complex capital structures:

Table 2: Partition types and use cases.

Partition	Use Case
COMMON	Common shares with standard voting and dividend rights
PREFERRED_A	Series A preferred with liquidation preference and anti-dilution
PREFERRED_B	Series B preferred with distinct terms
LOCKUP_6M	Shares subject to 6-month lockup from issuance
LOCKUP_12M	Shares subject to 12-month lockup (Rule 144)
VESTING	Shares subject to vesting schedule (employee grants)
RESTRICTED	Shares subject to regulatory hold
CONVERTIBLE	Convertible instruments (notes, SAFEs) pending conversion
TREASURY	Issuer treasury shares (not outstanding, no voting rights)
ESOP	Employee stock option pool allocation

4.2 Partition Operations

Listing 4: Partition management operations.

```

1  contract SecurityToken is ISecurityToken {
2      mapping(bytes32 => mapping(address => uint256)) internal partitionBalances;
3      mapping(bytes32 => PartitionConfig) internal partitionConfigs;
4
5      struct PartitionConfig {
6          bool transferable;
7          uint256 lockupExpiry;           // 0 = no lockup
8          address complianceOracle;      // partition-specific oracle
9          uint256 maxHolders;            // 0 = unlimited
10         VestingSchedule vesting;       // empty = no vesting
11         bool convertible;
12         bytes32 convertTarget;          // target partition for conversion
13         uint256 convertRatio;           // conversion ratio (basis points)
14     }
15
16     struct VestingSchedule {
17         uint256 cliffDuration;
18         uint256 totalDuration;
19         uint256 startTime;
20     }
21
22     function transferByPartition(
23         bytes32 partition, address from, address to,
24         uint256 value, bytes calldata data
25     ) external override returns (bytes32) {
26         PartitionConfig memory config = partitionConfigs[partition];
27
28         // Lockup check
29         require(
30             config.lockupExpiry == 0 || block.timestamp >= config.lockupExpiry,
31             "partition locked"
32         );
33
34         // Vesting check
35         if (config.vesting.totalDuration > 0) {
36             uint256 vested = vestedAmount(partition, from);
37             require(value <= vested - partitionTransferred[partition][from],
38                 "exceeds vested amount");
39         }

```

```

40
41 // Compliance check via partition-specific or global oracle
42 address oracle = config.complianceOracle != address(0)
43 ? config.complianceOracle
44 : defaultComplianceOracle;
45
46 (bytes1 status,,) = ITransferRestriction(oracle)
47 .detectTransferRestriction(from, to, value);
48 require(status == 0x00, "transfer restricted");
49
50 // Execute transfer
51 partitionBalances[partition][from] -= value;
52 partitionBalances[partition][to] += value;
53
54 emit TransferByPartition(partition, from, to, value, data);
55 return partition;
56 }
57 }

```

5 Controller Operations with MPC

5.1 Threshold Controller

In the original Arca `st-contracts`, the controller was a single Ethereum address. If that key was compromised, the attacker could force-transfer any holder's securities. The Lux standard replaces the single controller with an MPC threshold controller.

Controller operations (forced transfers, redemptions, document updates) require a t -of- n threshold signature from a controller quorum. The default configuration uses 2-of-3:

1. **Issuer share:** Held by the securities issuer.
2. **Platform share:** Held by the ATS operator [2].
3. **Regulatory share:** Held by the regulatory custodian or compliance officer.

Listing 5: MPC-controlled forced transfer.

```

1 contract SecurityToken is ISecurityToken {
2     address public mpcController; // MPC threshold address
3
4     modifier onlyController() {
5         require(msg.sender == mpcController, "not controller");
6         -;
7     }
8
9     function controllerTransfer(
10         address from, address to, uint256 value,
11         bytes calldata data, bytes calldata operatorData
12     ) external override onlyController {
13         // Controller can transfer regardless of restrictions
14         // Used for: court orders, regulatory seizures, error correction
15         _transfer(from, to, value);
16         emit ControllerTransfer(msg.sender, from, to, value, data, operatorData);
17     }
18
19     function controllerRedeem(
20         address holder, uint256 value,
21         bytes calldata data, bytes calldata operatorData
22     ) external override onlyController {
23         _burn(holder, value);
24         emit ControllerRedemption(msg.sender, holder, value, data, operatorData);
25     }
26 }

```

The `mpcController` address is the public key derived from the distributed key generation ceremony. No single party ever possesses the full private key.

5.2 Controller Action Types

Table 3: Controller actions and their legal bases.

Action	Legal Basis
Forced transfer	Court order, regulatory directive
Forced redemption	Issuer buyback, regulatory delisting
Account freeze	SAR investigation, OFAC designation
Document update	Prospectus amendment, offering circular update
Partition migration	Corporate action (merger, conversion)
Emergency pause	Material adverse event, security incident

6 Post-Quantum Security

6.1 Threat Model

Securities tokens are long-lived assets. A bond may have a 30-year maturity. A common stock may exist indefinitely. The ECDSA signatures used in standard EVM are vulnerable to Shor’s algorithm, which can derive private keys from public keys in polynomial time on a cryptographically relevant quantum computer.

For securities tokens, the threat is not abstract. An attacker who records current ECDSA public keys can, once a quantum computer is available, derive the corresponding private keys and forge transfers. This “harvest now, decrypt later” attack is particularly dangerous for long-lived securities.

6.2 Lux PQ Precompiles

The Lux EVM includes precompiled contracts for post-quantum signature verification:

- **ETHFALCON** (precompile 0x0301): Falcon-512 signature verification, providing 128-bit post-quantum security [5].
- **Ringtail** (precompile 0x0302): Hybrid classical/post-quantum signature verification combining ECDSA with Dilithium [1].

Listing 6: Post-quantum transfer verification.

```
1 contract PQSecurityToken is SecurityToken {
2     // Precompile addresses
3     address constant ETHFALCON = address(0x0301);
4     address constant RINGTAIL = address(0x0302);
5
6     bool public pqRequired; // enforce PQ signatures
7
8     function transferByPartitionPQ(
9         bytes32 partition, address from, address to,
10        uint256 value, bytes calldata data,
11        bytes calldata pqSignature
12    ) external returns (bytes32) {
13        if (pqRequired) {
14            bytes32 msgHash = keccak256(abi.encodePacked(
15                partition, from, to, value, block.chainid, nonces[from]++
16            ));
17            (bool ok,) = RINGTAIL.staticcall(
18                abi.encodePacked(msgHash, pqSignature, pqPublicKeys[from])
19            );
20            require(ok, "PQ signature invalid");
21        }
22    }
```

```

23     return transferByPartition(partition, from, to, value, data);
24 }
25 }

```

6.3 Migration Path

Existing securities tokens can migrate to post-quantum security incrementally:

1. **Phase 1:** Deploy PQ-aware token contracts alongside classical contracts.
2. **Phase 2:** Investors register PQ public keys via I-Chain DID updates.
3. **Phase 3:** Enable optional PQ signature verification for transfers.
4. **Phase 4:** Require PQ signatures for all transfers (set `pqRequired = true`).
5. **Phase 5:** Rotate controller MPC shares to PQ-safe threshold scheme.

7 Comparison with Existing Standards

Table 4: Security token standard comparison.

Feature	Lux	ERC-3643	ERC-1400	ERC-1404
Partitioned holdings	Yes	No	Yes	No
Dynamic compliance oracle	Yes	Yes (ONCHAINID)	No	No
MPC controller	Yes	No	No	No
Post-quantum signatures	Yes	No	No	No
Forced transfers	Yes	Yes	Yes	No
Document management	Yes	No	Yes	No
DID-based identity	Yes	Yes (ONCHAINID)	No	No
Hierarchical partitions	Yes	No	Partial	No
Vesting schedules	Yes	No	No	No
Convertible instruments	Yes	No	No	No

7.1 ERC-3643 (T-REX)

ERC-3643, the Token for Regulated Exchanges (T-REX) standard, is the closest competitor to the Lux standard. Both use on-chain identity registries for compliance checking. Key differences:

- T-REX uses ONCHAINID, a centralized identity provider. Lux uses I-Chain DIDs with verifiable credentials from multiple providers.
- T-REX has a single controller address. Lux uses MPC threshold controllers.
- T-REX does not support partitioned holdings or post-quantum signatures.
- T-REX is deployed on Ethereum mainnet with high gas costs. Lux EVM provides sub-cent transfer costs.

8 Document Management

Securities require extensive documentation: prospectuses, offering circulars, subscription agreements, financial statements, and regulatory filings. The Lux standard stores document references on-chain with content hashes for integrity verification:

Listing 7: Document management.

```

1 contract SecurityToken is ISecurityToken {
2     struct Document {
3         string uri;           // IPFS or HTTPS URI
4         bytes32 hash;         // SHA-256 of document content
5         uint256 timestamp;

```

```

6      address updatedBy; // must be MPC controller
7  }
8
9  mapping(bytes32 => Document) internal documents;
10 bytes32[] internal documentNames;
11
12 function setDocument(
13     bytes32 name, string calldata uri, bytes32 hash
14 ) external override onlyController {
15     documents[name] = Document(uri, hash, block.timestamp, msg.sender);
16     if (!documentExists[name]) {
17         documentNames.push(name);
18         documentExists[name] = true;
19     }
20     emit DocumentUpdated(name, uri, hash);
21 }
22 }

```

Standard document names include:

- PROSPECTUS: The offering prospectus or offering circular.
- SUBSCRIPTION: The subscription agreement template.
- PPM: Private placement memorandum.
- FINANCIALS: Most recent audited financial statements.
- LEGAL_OPINION: Legal opinion on token characterization.
- TAX_OPINION: Tax opinion on token treatment.
- TRANSFER_RESTRICTIONS: Summary of applicable transfer restrictions.

9 Gas Cost Analysis

Table 5: Gas costs for security token operations on Lux EVM vs Ethereum.

Operation	Gas (Lux)	Gas (ETH)	Cost (Lux)*
Simple transfer	65,000	65,000	\$0.003
Partition transfer	85,000	85,000	\$0.004
Transfer with compliance check	120,000	120,000	\$0.006
Partition transfer + PQ verify	180,000	N/A	\$0.009
Issue tokens	95,000	95,000	\$0.005
Controller forced transfer	75,000	75,000	\$0.004
Set document	55,000	55,000	\$0.003

*At Lux base gas price of 25 nLUX per gas, LUX price \$2.00.

10 Conclusion

The Lux Security Token Standard represents the natural evolution of the Arca `st-contracts` architecture from 2018. By combining ERC-1400 partitioned security tokens with dynamic compliance oracles, MPC threshold controllers, and post-quantum cryptographic precompiles, the standard eliminates the security and architectural limitations of first-generation security token implementations while preserving full regulatory compliance.

The standard is deployed on Lux EVM and serves as the token layer for the non-custodial ATS described in [2]. Transfer restrictions are enforced in coordination with the transfer agent registry [4], and securities are routable through the smart order router [3].

References

- [1] Z. Kelling, V. Seesahai, “M-Chain: Decentralized Multi-Party Computation Custody with Quantum-Safe Threshold Signatures,” Lux Partners, 2023.
- [2] Z. Kelling, “Non-Custodial Blockchain ATS: Architecture of a Regulatory-Compliant Securities Exchange,” Lux Partners, 2025.
- [3] Z. Kelling, “Smart Order Routing for Digital Securities: Multi-Provider Execution on Lux Network,” Lux Partners, 2025.
- [4] Z. Kelling, “Blockchain Transfer Agent: On-Chain Cap Table with MPC-Custodied Registry,” Lux Partners, 2025.
- [5] Z. Kelling, “ETHFALCON: Post-Quantum Signature Verification as EVM Precompile,” Lux Partners, 2024.
- [6] Z. Kelling, “Lux Credit Protocol Specification,” Lux Partners, 2023.
- [7] A. Dossa, P. Ruiz, F. Vogelsteller, S. Gosselin, “ERC-1400: Security Token Standard,” Ethereum Improvement Proposals, 2018.
- [8] R. Bose, J. Cawley, “ERC-1404: Simple Restricted Token Standard,” Ethereum Improvement Proposals, 2018.
- [9] J. Music, R. Music, “ERC-3643: T-REX — Token for Regulated Exchanges,” Ethereum Improvement Proposals, 2021.
- [10] R. Gennaro, S. Goldfeder, “One Round Threshold ECDSA with Identifiable Abort,” Cryptology ePrint Archive, 2020.
- [11] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” FOCS, 1994.
- [12] National Institute of Standards and Technology, “Post-Quantum Cryptography Standardization,” NIST, 2024.